

Coding Assignment -1

Name : Mohammad Ibran

Section : FB-37

Roll no : 12515000953

Subject : DSA (BCSC 1006)

Q1. Maximum Product to Two Element in Array

```
1 class Solution {
2     public int maxProduct(int[] nums) {
3         int max1 = 0;
4         int max2 = 0;
5         for (int num : nums){
6             if (num > max1){
7
8                 max2 = max1;
9                 max1 = num;
10            }else if (num > max2){
11                max2 = num;
12            }
13        }
14        return (max1-1)*(max2-1);
15    }
16 }
```

Q2. Second-Largest.

```
1 int getSecondLargest(int *arr, int n) {
2     // code here
3
4     int largest = -1;
5     int secondLargest = -1;
6
7     for (int i = 0; i < n; i++) {
8         if (arr[i] > largest) {
9             secondLargest = largest;
10            largest = arr[i];
11        }
12        else if (arr[i] > secondLargest && arr[i] != largest) {
13            secondLargest = arr[i];
14        }
15    }
16    return secondLargest;
17 }
18
19
20 }
```

Q3. Find Peak Element.

```

1 class Solution {
2     public int findPeakElement(int[] nums) {
3         int left = 0;
4         int right = nums.length - 1;
5         while(left < right){
6             int mid = left + (right - left) / 2;
7             if (nums[mid] < nums[mid + 1]){
8                 left = mid + 1;
9             } else {
10                right = mid;
11            }
12        }
13        return left;
14    }
15 }

```

Q4. Search in Rotated Sorted Array.

```

1 class Solution {
2     public int search(int[] nums, int target) {
3         int start = 0;
4         int end = nums.length - 1;
5
6         while(start <= end){
7             int mid = (start + end) / 2;
8
9             if (nums[mid] == target){
10                return mid;
11            } else if (nums[start] <= nums[mid]){
12
13                if (target >= nums[start] && target < nums[mid]){
14                    end = mid - 1;
15                } else {
16                    start = mid + 1;
17                }
18            } else {
19                if (target > nums[mid] && target <= nums[end]){
20                    start = mid + 1;
21                } else {
22                    end = mid - 1;
23                }
24            }
25        }
26        return -1;
27    }
28 }
29
30
31 }

```

Q5. Fruits Into Baskets II

```

1 class Solution {
2     public int numOfUnplacedFruits(int[] fruits, int[] baskets) {
3
4         int n = fruits.length;
5         int unplaced = 0;
6
7         for (int i = 0; i < n; i++) {
8             boolean placed = false;
9
10            for (int j = 0; j < n; j++) {
11                if (baskets[j] >= fruits[i]) {
12                    baskets[j] = -1; // basket used
13                    placed = true;
14                    break;
15                }
16            }
17
18            if (!placed) {
19                unplaced++;
20            }
21        }
22        return unplaced;
23    }
24 }
25
26
27 }

```

Q6. Best Time to Buy and Sell Stock.

```

1 class Solution {
2     public int maxProfit(int[] prices) {
3         int minPrice = prices[0];
4         int maxProfit = 0;
5
6         for (int i = 1; i < prices.length; i++) {
7             if (prices[i] < minPrice) {
8                 minPrice = prices[i];
9             }
10
11             int profit = prices[i] - minPrice;
12
13             if (profit > maxProfit) {
14                 maxProfit = profit;
15             }
16         }
17
18         return maxProfit;
19     }
20 }

```

Q7. Two Sum.

```

1 class Solution {
2     public int[] twoSum(int[] nums, int target) {
3
4         for(int i = 0; i<nums.length; i++){
5             for(int j=i+1; j<nums.length; j++){
6                 if(nums[i]+nums[j]== target){
7                     return new int[]{i,j};
8                 }
9             }
10        }
11        return new int[]{};
12    }
13 }
14 }

```

Q8. How Many Number Are Smaller Than the Current Number.

```

1 class Solution {
2     public int[] smallerNumbersThanCurrent(int[] nums) {
3
4         int n = nums.length;
5         int[] ans = new int[n];
6
7         for (int i = 0; i < n; i++) {
8             int count = 0;
9
10            for (int j = 0; j < n; j++) {
11                if (nums[j] < nums[i]) {
12                    count++;
13                }
14            }
15
16            ans[i] = count;
17        }
18
19        return ans;
20    }
21 }
22 }

```

Q9. Maximum Product of Three Number.

```

1 class Solution {
2     public int maximumProduct(int[] nums) {
3         Arrays.sort(nums);
4         int n = nums.length;
5         int p1 = nums[n-1] * nums[n-2] * nums[n-3];
6         int p2 = nums[0] * nums[1] * nums[n-1];
7
8         return Math.max(p1,p2);
9     }
10 }

```

Q10. Remove Duplicates form Sorted Array.

```

1 class Solution {
2     public int removeDuplicates(int[] nums) {
3         int j = 1;
4         for (int i = 1; i < nums.length; i++) {
5             if (nums[i] != nums[i - 1]) {
6                 nums[j] = nums[i];
7                 j++;
8             }
9         }
10        return j;
11    }
12 }
13 }

```